

Experiment No. 8

Experiment Name - Customer Churn Prediction using Decision Tree Classification

NAME : UTSAV KUMAR KUSHWAHA

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 63

CU ID : CU240251668

Aim

To build and evaluate a Decision Tree Classification model for predicting customer churn using a real-world dataset and analyze the factors influencing customer retention.

Purpose

Customer churn prediction is a critical application of supervised machine learning that helps organizations identify customers who are likely to discontinue a service or product. Early identification of such customers enables businesses to implement targeted retention strategies, improve customer satisfaction, and reduce revenue loss. In this experiment, the Telco Customer Churn Dataset is used to develop a Decision Tree classifier, evaluate its performance, and interpret the generated decision rules.

Steps

1. Select and download a suitable dataset (Telco Customer Churn Dataset from Kaggle).

```
from google.colab import files
uploaded = files.upload() # select Telco-Customer-Churn.csv when prompted
import os
print(os.listdir())
Choose Files
Telco-Customer-Churn.csv
Telco-Customer-Churn.csv - 977501 bytes, last modified: 9/9/2026 - 100% done
Saving Telco-Customer-Churn.csv to Telco-Customer-Churn.csv
['.config', 'Telco-Customer-Churn.csv', 'sample_data']
```

2. Import the required Python libraries and load the dataset into a Pandas DataFrame.

```
# Importing the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score, f1_score)
```

```
print("All required libraries imported successfully!")
```

```
# Load the dataset
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
print("\nFirst 5 rows:")
display(df.head())
```

```
All required libraries imported successfully!
Dataset loaded successfully!
Shape of dataset: (7043, 21)
```

First 5 rows:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	

	No	No	No	Month-to-month	Yes
	PaymentMethod	MonthlyCharges	TotalCharges	Churn	
0	Electronic check	29.85	29.85	No	
1	Mailed check	56.95	1889.5	No	
2	Mailed check	53.85	108.15	Yes	
3	Bank transfer (automatic)	42.30	1840.75	No	
4	Electronic check	70.70	151.65	Yes	

3. Perform data preprocessing by handling missing values, encoding categorical variables, and selecting relevant features.

```
# Basic inspection
print("Missing values:")
print(df.isnull().sum())
print("\nData types:")
print(df.dtypes)
print("\nChurn value counts:")
print(df['Churn'].value_counts())

# TotalCharges is object type because of empty strings → convert to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
print("\nMissing values after conversion:", df['TotalCharges'].isnull().sum())

# Drop rows with missing TotalCharges (only 11 rows)
df.dropna(subset=['TotalCharges'], inplace=True)
df.reset_index(drop=True, inplace=True)

# Drop customerID (not useful for prediction)
df.drop('customerID', axis=1, inplace=True)

# Encode target variable
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# Label Encode all remaining categorical columns
cat_cols = df.select_dtypes(include='object').columns.tolist()
le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col])

print("\nShape after preprocessing:", df.shape)
print("\nEncoded sample:")
display(df.head())
print("\nFeature list:", df.columns.tolist())

Missing values:
customerID      0
gender           0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64

Churn value counts:
No      5174
Yes     1869
Name: Churn, dtype: int64

Missing values after conversion: 11
Shape after preprocessing: (7032, 20)
```

```
Encoded sample:
gender SeniorCitizen Partner Dependents tenure PhoneService \
0      0             0      1           0       1           0
1      1             0      0           0      34           1
2      1             0      0           0       2           1
3      1             0      0           0      45           0
4      0             0      0           0       2           1

MultipleLines InternetService OnlineSecurity OnlineBackup \
0             1              0              0             2
1             0              0              2             0
2             0              0              2             2
```

3	1	0	2	0	
4	0	1	0	0	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	0	0	0	0	0	
1	2	0	0	0	1	
2	0	0	0	0	0	
3	2	2	0	0	1	
4	0	0	0	0	0	

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	1	2	29.85	29.85	0
1	0	3	56.95	1889.50	0
2	1	3	53.85	108.15	1
3	0	0	42.30	1840.75	0
4	1	2	70.70	151.65	1

```
Feature list: ['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod',
'MonthlyCharges', 'TotalCharges', 'Churn']
```

4. Split the dataset into training and testing sets (80:20 ratio).

```
# Separate features and target
X = df.drop('Churn', axis=1)
y = df['Churn']

# Train-Test Split (80:20) with stratification to preserve class ratio
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Training set shape :", X_train.shape)
print("Testing set shape  :", X_test.shape)
print("\nChurn distribution in training set:")
print(y_train.value_counts(normalize=True).round(3))
print("\nChurn distribution in testing set:")
print(y_test.value_counts(normalize=True).round(3))

Training set shape : (5625, 19)
Testing set shape  : (1407, 19)

Churn distribution in training set:
0    0.734
1    0.266
Name: Churn, dtype: float64

Churn distribution in testing set:
0    0.734
1    0.266
Name: Churn, dtype: float64
```

5. Train a Decision Tree Classifier with suitable parameters (max_depth, criterion).

```
# Train Decision Tree Classifier
# Using Gini Index and limiting depth to reduce overfitting
dt_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_split=20,
    min_samples_leaf=10,
    random_state=42
)

dt_model.fit(X_train, y_train)
print("Decision Tree model trained successfully!")
print("Tree depth      :", dt_model.get_depth())
print("Number of leaves  :", dt_model.get_n_leaves())
print("Number of features :", dt_model.n_features_in_)

Decision Tree model trained successfully!
Tree depth      : 5
Number of leaves : 27
Number of features : 19
```

6. Use the trained model to predict customer churn on the testing dataset.

```
# Make predictions
y_pred = dt_model.predict(X_test)
y_pred_proba = dt_model.predict_proba(X_test)[:, 1]

print("Predictions completed!")
print("\nSample of actual vs predicted (first 15):")
comparison = pd.DataFrame({
    'Actual': y_test.values[:15],
    'Predicted': y_pred[:15]
})
display(comparison)
Predictions completed!
```

Sample of actual vs predicted (first 15):

	Actual	Predicted
0	0	0
1	0	0
2	1	1
3	0	0
4	0	0
5	1	0
6	0	0
7	1	1
8	0	0
9	0	0
10	1	1
11	0	0
12	0	0
13	1	1
14	0	0

7. Evaluate the model using Confusion Matrix, Accuracy, Precision, Recall and F1-Score.

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

# Metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\n----- Performance Metrics -----")
print(f"Accuracy : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall : {rec:.4f}")
print(f"F1-Score : {f1:.4f}")

print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=['No Churn', 'Churn']))

# Visualize Confusion Matrix
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Churn', 'Churn'],
            yticklabels=['No Churn', 'Churn'])
plt.title('Confusion Matrix - Decision Tree')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
Confusion Matrix:
[[930 103]
 [178 196]]

----- Performance Metrics -----
Accuracy : 0.8003
Precision : 0.6555
Recall : 0.5241
F1-Score : 0.5825

Detailed Classification Report:

```

	precision	recall	f1-score	support
No Churn	0.84	0.90	0.87	1033
Churn	0.66	0.52	0.58	374
accuracy			0.80	1407
macro avg	0.75	0.71	0.73	1407
weighted avg	0.79	0.80	0.79	1407

8. Visualize the Decision Tree and identify the most influential features.

```
# Feature Importance
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("Top 10 Most Important Features:")
display(importance.head(10))

# Plot Feature Importance
plt.figure(figsize=(9, 5))
sns.barplot(data=importance.head(10), x='Importance', y='Feature', palette='viridis')
plt.title('Top 10 Feature Importances (Decision Tree)')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()

# Visualize the Decision Tree (simplified view)
```

```
plt.figure(figsize=(18, 10))
plot_tree(dt_model, feature_names=X.columns, class_names=['No Churn', 'Churn'],
          filled=True, rounded=True, fontsize=7, max_depth=3)
plt.title('Decision Tree (Depth limited to 3 for readability)')
plt.show()
print("Decision Tree visualization completed!")

Top 10 Most Important Features:
      Feature Importance
14   Contract    0.4821
4      tenure    0.1789
17  MonthlyCharges 0.1124
18   TotalCharges 0.0678
8   OnlineSecurity 0.0456
15 PaperlessBilling 0.0312
7   InternetService 0.0289
11    TechSupport  0.0214
16   PaymentMethod 0.0156
2      Partner     0.0089
```

[Bar chart of feature importances + Decision Tree diagram]

Decision Tree visualization completed!

9. Analyze the classification results and determine key factors contributing to customer churn.

```
print("=" * 70)
print("KEY FACTORS CONTRIBUTING TO CUSTOMER CHURN")
print("=" * 70)
print("1. Contract Type (Highest Importance ~48%)")
print("  - Month-to-month contracts have much higher churn.")
print("  - One-year / Two-year contracts show strong retention.")
print("2. Tenure (Importance ~18%)")
print("  - New customers (low tenure) are far more likely to churn.")
print("  - Long-tenure customers are loyal.")
print("3. Monthly Charges (Importance ~11%)")
print("  - Higher monthly charges increase churn probability.")
print("4. Online Security & Tech Support reduce churn.")
print("5. Paperless Billing & Payment Method also influence churn.")
print("Business Interpretation:")
print("Contract commitment and early customer experience are the")
print("strongest drivers of churn. Pricing and value-added services matter.")
=====
KEY FACTORS CONTRIBUTING TO CUSTOMER CHURN
=====
```

1. Contract Type (Highest Importance ~48%)
 - Month-to-month contracts have much higher churn.
 - One-year / Two-year contracts show strong retention.
2. Tenure (Importance ~18%)
 - New customers (low tenure) are far more likely to churn.
 - Long-tenure customers are loyal.
3. Monthly Charges (Importance ~11%)
 - Higher monthly charges increase churn probability, especially when not paired with long-term contracts.
4. Online Security & Tech Support
 - Customers without these add-on services churn more.
 - Presence of security/support services improves retention.
5. Paperless Billing & Payment Method
 - Electronic check / paperless billing customers show slightly higher churn tendency.

Business Interpretation:
 The model clearly shows that contract commitment and early customer experience (first few months) are the strongest drivers of churn. Pricing and value-added services also matter.

10. Summarize the findings and recommend business strategies for improving customer retention.

```
print("=" * 70)
print("FINAL SUMMARY & BUSINESS RECOMMENDATIONS")
print("=" * 70)

print('')
Model Performance Summary
-----
Algorithm      : Decision Tree Classifier (Gini, max_depth=5)
Accuracy       : 80.03%
Precision (Churn) : 65.55%
Recall (Churn)  : 52.41%
F1-Score       : 58.25%

Key Insights
-----
- Contract type is the single strongest predictor of churn.
- Customers in their first 6-12 months are at highest risk.
```

- Higher monthly charges without long-term commitment increase risk.
- Value-added services (Security, Tech Support) reduce churn.

Recommended Business Strategies

1. Push longer-term contracts with attractive discounts or benefits for month-to-month customers.
2. Create a dedicated onboarding & early-engagement program for new customers (first 90 days).
3. Bundle Online Security and Tech Support at discounted rates for high-risk segments.
4. Offer loyalty rewards / usage-based discounts to high monthly charge customers who are still on month-to-month plans.
5. Use the model scores to prioritize outbound retention calls for customers with high predicted churn probability.

Save the model predictions for further use

```
results = X_test.copy()
results['Actual_Churn'] = y_test.values
results['Predicted_Churn'] = y_pred
results.to_csv("Churn_Predictions.csv", index=False)
print("Predictions saved to 'Churn_Predictions.csv'")
```

FINAL SUMMARY & BUSINESS RECOMMENDATIONS

Model Performance Summary

```
Algorithm      : Decision Tree Classifier (Gini, max_depth=5)
Accuracy       : 80.03%
Precision (Churn) : 65.55%
Recall (Churn)  : 52.41%
F1-Score       : 58.25%
```

Key Insights

- Contract type is the single strongest predictor of churn.
- Customers in their first 6-12 months are at highest risk.
- Higher monthly charges without long-term commitment increase risk.
- Value-added services (Security, Tech Support) reduce churn.

Recommended Business Strategies

1. Push longer-term contracts with attractive discounts or benefits for month-to-month customers.
2. Create a dedicated onboarding & early-engagement program for new customers (first 90 days).
3. Bundle Online Security and Tech Support at discounted rates for high-risk segments.
4. Offer loyalty rewards / usage-based discounts to high monthly charge customers who are still on month-to-month plans.
5. Use the model scores to prioritize outbound retention calls for customers with high predicted churn probability.

Predictions saved to 'Churn_Predictions.csv'

Questions & Answers

1. What is classification, and how does it differ from regression?

Classification is a supervised machine learning task in which the model learns to assign input data points to one of a set of predefined discrete categories (classes). The output is categorical (e.g., Yes/No, Churn/No-Churn, Spam/Not-Spam).

Difference from Regression:

- **Classification** predicts discrete class labels. Example: Will the customer churn? (Yes or No).
- **Regression** predicts continuous numerical values. Example: What will be the customer's monthly charges next month?
- Evaluation metrics also differ: classification uses Accuracy, Precision, Recall, F1-Score, while regression uses MAE, MSE, RMSE, R^2 .

2. Explain the working principle of the Decision Tree algorithm.

A Decision Tree is a flowchart-like structure where each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a class label (or a probability distribution).

Working steps:

1. Start with the entire training dataset at the root.
2. Select the best feature to split on using a criterion such as Gini Index or Information Gain (Entropy).
3. Create child nodes for each possible outcome of the selected feature.
4. Recursively repeat the process on each child node until a stopping condition is met (pure node, maximum depth reached, or minimum samples remaining).
5. Assign the majority class (or class probabilities) to the leaf nodes.

At prediction time, a new instance follows the path from root to leaf according to the feature tests and receives the class of that leaf.

3. Differentiate between Gini Index and Entropy as splitting criteria.

Aspect	Gini Index	Entropy (Information Gain)
Formula	$Gini = 1 - \sum (p_i)^2$	$Entropy = - \sum p_i \log_2(p_i)$
Range	0 (pure) to 0.5 (binary equal classes)	0 (pure) to 1 (binary equal classes)
Interpretation	Measures impurity / probability of misclassification.	Measures disorder or uncertainty in the node.
Computation	Faster (no logarithm).	Slightly slower (involves log).
Preference	Default in scikit-learn CART implementation; often preferred for speed.	Used in ID3 / C4.5; theoretically more information-theoretic.
Result	Both usually produce very similar trees in practice.	Both usually produce very similar trees in practice.

4. What is a Confusion Matrix? Explain its components.

A Confusion Matrix is a table that summarizes the performance of a classification model by comparing the actual labels with the predicted labels. For a binary classification problem it is a 2x2 matrix:

- **True Positive (TP)** – Actual Churn predicted as Churn.
- **True Negative (TN)** – Actual No-Churn predicted as No-Churn.
- **False Positive (FP)** – Actual No-Churn predicted as Churn (Type I error).
- **False Negative (FN)** – Actual Churn predicted as No-Churn (Type II error).

In this experiment the confusion matrix was: [[930, 103], [178, 196]] → TN=930, FP=103, FN=178, TP=196.

5. Define Accuracy, Precision, Recall, and F1-Score. Why are these metrics important?

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$ — overall correctness of the model.
- **Precision** = $TP / (TP + FP)$ — of all predicted churn cases, how many were actually churn.
- **Recall (Sensitivity)** = $TP / (TP + FN)$ — of all actual churn cases, how many the model correctly identified.
- **F1-Score** = $2 \times (Precision \times Recall) / (Precision + Recall)$ — harmonic mean that balances Precision and Recall.

These metrics are important because Accuracy alone can be misleading on imbalanced datasets (here ~73% No-Churn). Precision tells us how trustworthy the positive predictions are, Recall tells us how many actual churners we catch, and F1 gives a single balanced measure. In churn prediction, high Recall is often desirable so that fewer at-risk customers are missed.

6. What is overfitting in a Decision Tree? How can it be reduced?

Overfitting occurs when the tree grows too deep and learns noise or very specific patterns from the training data that do not generalize to new data. The training accuracy becomes very high while test accuracy drops.

Ways to reduce overfitting:

- Limit maximum depth (max_depth).
- Set minimum number of samples required to split a node (min_samples_split).
- Set minimum number of samples required at a leaf (min_samples_leaf).
- Use pruning techniques (pre-pruning or post-pruning).
- Apply ensemble methods such as Random Forest or Gradient Boosting.

In this experiment max_depth=5, min_samples_split=20 and min_samples_leaf=10 were used to control overfitting.

7. Why is customer churn prediction important for businesses?

Acquiring a new customer typically costs 5–7 times more than retaining an existing one. Churn prediction allows a company to:

- Identify customers who are likely to leave before they actually leave.
- Launch targeted retention offers, discounts or personalized communication.
- Reduce revenue leakage and improve Customer Lifetime Value (CLV).
- Allocate marketing and support resources more efficiently.
- Gain insights into the root causes of dissatisfaction (contract type, pricing, service quality, etc.).

Even a small reduction in churn rate can produce a large increase in profitability.

8. What are the advantages and limitations of the Decision Tree algorithm?

Advantages:

- Easy to understand and interpret (white-box model); rules can be visualized.
- Requires little data preparation (no feature scaling needed).
- Can handle both numerical and categorical features.
- Captures non-linear relationships and feature interactions automatically.
- Fast to train and predict on medium-sized datasets.

Limitations:

- Prone to overfitting if not properly regularized.
- Unstable — small changes in data can produce a completely different tree.
- Biased toward features with more levels (when using certain splitting criteria).
- Generally lower predictive accuracy than ensemble methods (Random Forest, XGBoost).
- Can create biased trees if the dataset is highly imbalanced.

9. Mention three real-world applications of Decision Tree Classification other than customer churn prediction.

1. **Medical Diagnosis** – Predicting disease presence (e.g., diabetes, heart disease) based on patient symptoms and lab results.
2. **Credit Risk / Loan Approval** – Deciding whether to approve a loan application based on applicant's income, credit history, employment status, etc.
3. **Fraud Detection** – Identifying potentially fraudulent credit-card or insurance claims by learning patterns from historical transaction data.

Other common uses include spam email filtering, customer segmentation support, and equipment failure prediction in manufacturing.

10. How can the insights obtained from a churn prediction model help organizations improve customer retention and business profitability?

Insights from a well-built churn model can be turned into concrete actions:

- **Targeted Retention Campaigns** – Focus expensive retention offers only on customers with high predicted churn probability, maximizing ROI.
- **Product & Pricing Optimization** – Discover that month-to-month contracts and high monthly charges drive churn → design better long-term plans and value bundles.
- **Service Improvement** – If lack of Tech Support or Online Security appears important, invest in those services or promote them more aggressively.
- **Early Warning System** – Score every customer regularly and trigger automated alerts or outreach when risk rises.
- **Resource Allocation** – Sales and support teams can prioritize high-value, high-risk customers.

Collectively these actions reduce churn rate, increase average customer lifetime, lower acquisition cost pressure, and therefore improve overall profitability and competitive position.